Python Library Documentation: module textwrap
NAME
textwrap - Text wrapping and filling.
MODULE REFERENCE
https://docs.python.org/3.14/library/textwrap.html
The following documentation is automatically generated from the Python
source files. It may be incomplete, incorrect or include features that
are considered implementation detail and may vary between Python
implementations. When in doubt, consult the module reference at the
location listed above.
CLASSES
builtins.object
TextWrapper
class TextWrapper(builtins.object)
| TextWrapper(
| width=70,
| initial_indent='',
| subsequent_indent='',
| expand_tabs=True,
| replace_whitespace=True,
| fix_sentence_endings=False,
| break_long_words=True,
| drop_whitespace=True,
| break_on_hyphens=True,
| tabsize=8,
| *,
| max_lines=None,
| placeholder=' [...]'
|)
|
| Object for wrapping/filling text. The public interface consists of
| the wrap() and fill() methods; the other methods are just there for
| subclasses to override in order to tweak the default behaviour.
| If you want to completely replace the main wrapping algorithm,
| you'll probably have to override _wrap_chunks().
|
| Several instance attributes control various aspects of wrapping:
| width (default: 70)
| the maximum width of wrapped lines (unless break_long_words
| is false)
| initial_indent (default: "")
| string that will be prepended to the first line of wrapped
| output. Counts towards the line's width.
| subsequent_indent (default: "")
| string that will be prepended to all lines save the first
| of wrapped output; also counts towards each line's width.
| expand_tabs (default: true)
| Expand tabs in input text to spaces

before further processing.
| Each tab will become 0 .. 'tabsize' spaces, depending on its position
| in its line. If false, each tab is treated as a single character.
| tabsize (default: 8)
| Expand tabs in input text to 0 .. 'tabsize' spaces, unless
| 'expand_tabs' is false.
| replace_whitespace (default: true)
| Replace all whitespace characters in the input text by spaces
| after tab expansion. Note that if expand_tabs is false and
| replace_whitespace is true, every tab will be converted to a
| single space!
| fix_sentence_endings (default: false)
| Ensure that sentence-ending punctuation is always followed
| by two spaces. Off by default because the algorithm is
| (unavoidably) imperfect.
| break_long_words (default: true)
| Break words longer than 'width'. If false, those words will not
| be broken, and some lines might be longer than 'width'.
| break_on_hyphens (default: true)
| Allow breaking hyphenated words. If true, wrapping will occur
| preferably on whitespaces and right after hyphens part of
| compound words.
| drop_whitespace (default: true)
| Drop leading and trailing whitespace from lines.
| max_lines (default: None)
| Truncate wrapped lines.
| placeholder (default: ' [...]')
| Append to the last line of truncated text.
|
| Methods defined here:
|
| __init__(
| self,
| width=70,
| initial_indent='',
| subsequent_indent='',
| expand_tabs=True,
| replace_whitespace=True,
| fix_sentence_endings=False,
| break_long_words=True,
| drop_whitespace=True,
| break_on_hyphens=True,
| tabsize=8,
| *,
| max_lines=None,
| placeholder=' [...]'
|

)
| Initialize self. See help(type(self)) for accurate signature.
|
| fill(self, text)
| fill(text : string) -> string
|
| Reformat the single paragraph in 'text' to fit in lines of no
| more than 'self.width' columns, and return a new string
| containing the entire wrapped paragraph.
|
| wrap(self, text)
| wrap(text : string) -> [string]
|
| Reformat the single paragraph in 'text' so it fits in lines of
| no more than 'self.width' columns, and return a list of wrapped
| lines. Tabs in 'text' are expanded with string.expandtabs(),
| and all other whitespace characters (including newline) are
| converted to space.
|
| --
| Data descriptors defined here:
|
| __dict__
| dictionary for instance variables
|
| __weakref__
| list of weak references to the object
|
| --
| Data and other attributes defined here:
|
| sentence_end_re = re.compile('[a-z][\\.\\!\\?][\\"\\\']?\\z')
|
| unicode_whitespace_trans = {9: 32, 10: 32, 11: 32, 12: 32, 13: 32, 32:...
|
| wordsep_re = re.compile('\n (# any whitespace\n ...# word...
|
| wordsep_simple_re = re.compile('([\\\t\\\n\\\x0b\\\x0c\\\r\\ ]+)')
FUNCTIONS
dedent(text)
Remove any common leading whitespace from every line in 'text'.
This can be used to make triple-quoted strings line up with the left
edge of the display, while still presenting them in the source code
in indented form.
Note that tabs and spaces are both treated as whitespace, but they
are not equal: the lines " hello" and "\thello" are
considered to have no common leading whitespace.
Entirely